Student Name. _____ Roll No. _____

Department of Information Technology
Faculty of Engineering and Technology
University of Sindh, Jamshoro

BS (Data Science) Part-III
DASC- 523 Parallel and Distributed Computing

LAB # 5 – Reduction across threads for Parallel For loop synchronization

Lab Objective: To understand and work with the parallel reduction operators in OpenMP.

Description:

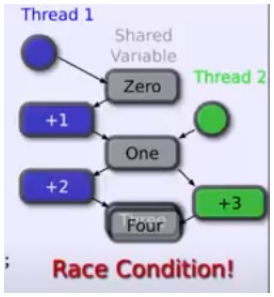
- In computer science, the reduction operator is a type of operator that is commonly used in parallel programming to reduce the elements of an array into a single result. Reduction operators are associative and often (but not necessarily) commutative. The reduction of sets of elements is an integral part of programming models such as Map Reduce, where a reduction operator is applied (mapped) to all elements before they are reduced. Other parallel algorithms use reduction operators as primary operations to solve more complex problems. Many reduction operators can be used for broadcasting to distribute data to all processors.

- A reduction operator can help break down a task into various partial tasks by calculating partial results which can be used to obtain a final result. It allows certain serial operations to be performed in parallel and the number of steps required for those operations to be reduced. A reduction operator stores the result of the partial tasks into a private copy of the variable. These private copies are then merged into a shared copy at the end.

- For example, if you write:

```
int sum=0;
#pragma omp parallel for
for (int i=0; i<N; i++)
  sum += i;
```

  You will find that the sum value depends on the number of threads, and is likely not the same as when you execute the code sequentially. The problem here is the race condition involving the sum variable, since this variable is shared between all threads.



Task - Write a parallel for loop that sums positive integers from 1 to 10. For dealing with race condition and producing correct output, employ reduction clause.

```
#include <omp.h>
#include <stdio.h>

int main()
{
int n = 10;
int total = 0;
   #pragma omp parallel for reduction (+: total)
   for (int i = 1; i <= n; i++) {
      total = total + i;
  }
      printf("The total is %d\n", total);
      return 0;
}
```